pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

Embeddings and a Vector Index

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: embeddings and a vector index

Three models, one 10-K

Chunks	1,158 at 512/64
Vectors	384 numbers each
MiniLM	the course's running model
Compared	bge-small, gte-small
Model	none today; nothing is sent

Today

1. An embedding; the committed matrix
2. Three embedding models
3. An exact index with NumPy
4. FAISS: the same chunks
5. An approximate index (HNSW)
6. Chroma: named, not installed
7. What an index was built from
8. Three models on the questions

Check every file you did not make; then measure.

How an embedding works

The problem	A question must be compared with every chunk, and text cannot be compared by arithmetic
The model	Reads the pieces through a small transformer, averages their states, scales the list to length 1
The dimension	384 numbers, the model's not the text's: two words and the longest chunk come out the same length
The score	The cosine of the angle between two lists: 1 for the same direction, near 0 for unrelated texts
You control	The model, its revision, and the batch size; what comes out is one float32 row per text, in order

An embedding, and the committed matrix

One chunk becomes 384 numbers

```
first five: [-0.023, -0.0462, -0.1087, -0.0153, -0.0383]
```

The cosine of two vectors is their score.

Every row has length 1: dot product = cosine.

Two checks on a file you did not make

1. SHA-256 of the chunk texts equals the record's

2. 50 chunks embedded live, within 1e-6

1,158
rows, one per chunk

384
columns

float32
committed

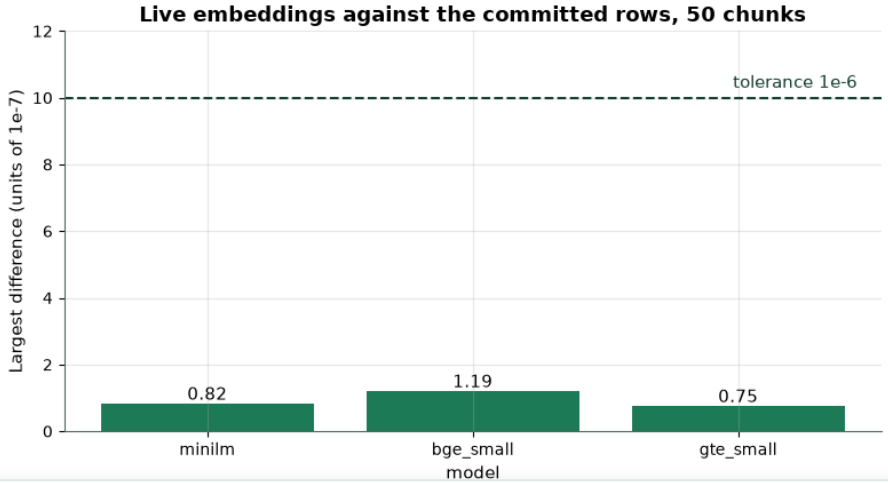
True
made from these chunks

Embedding a list once fixes its last bits; a live sample is not bit for bit the committed rows.

Three embedding models

Size, limit, and the live check

model	params (M)	piece limit	gap (1e-7)
minilm	22.71	256	0.82
bge_small	33.36	512	1.19
gte_small	33.36	512	0.75



bge-small's card (v1.5), read 2026-10-12

Query instruction: "Represent this sentence for searching relevant passages: " (questions only)

"No instruction only has a slight degradation in retrieval performance compared with using instruction."

No chunk passes MiniLM's 256 pieces at 512/64 (day 3): none of the three truncates anything.

An exact index: arc02 on MiniLM

Credit facility outstanding at December 31, 2025: the key is on page 37

```
rank 1: score 0.6802, boyd10k-7-036-0413, Item 7, page 36
rank 2: score 0.6375, boyd10k-1A-019-0225, Item 1A, page 19
rank 3: score 0.6273, boyd10k-8-082-0917, Item 8, page 82
```

The top chunk (quoted as published): holds 160.7: True; a page break inside: True

```
 the Credit Facility for

2024.

31Table of Contents

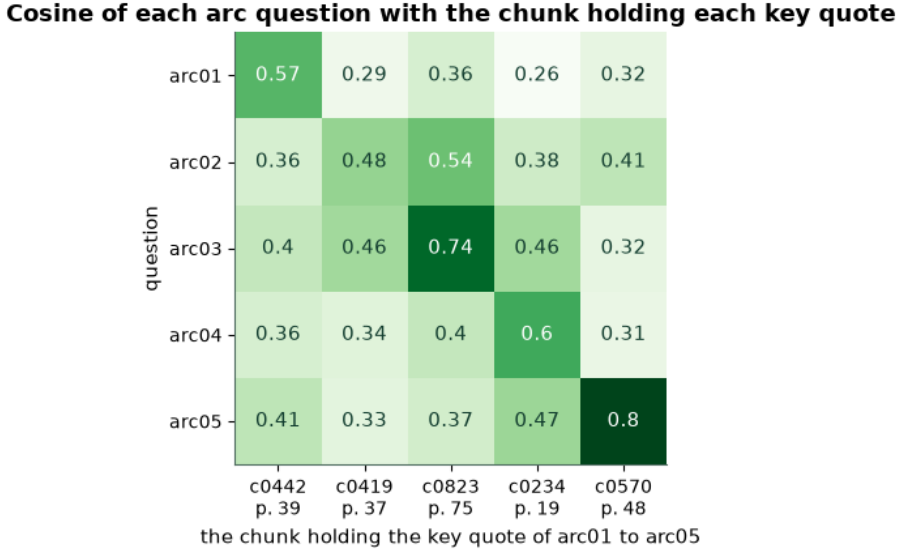
Indebtedness

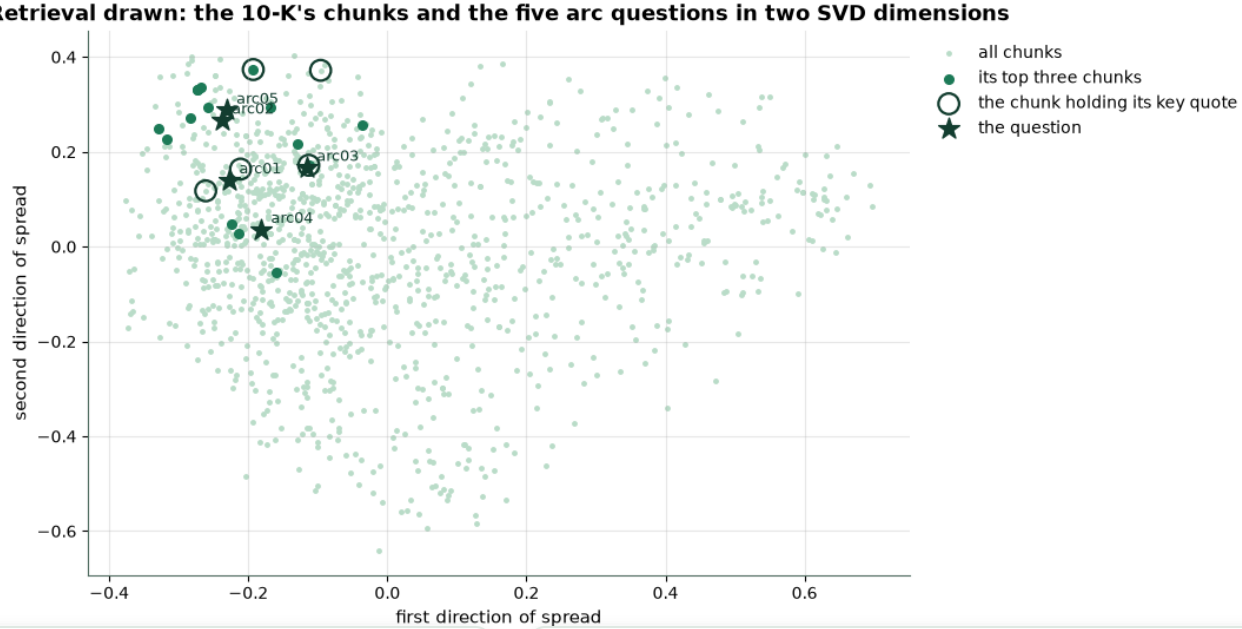
The outstanding principal balances of long-term debt, before unamortized discounts and fees, and the changes ...
```

Source: Boyd Gaming Corporation, Form 10-K for fiscal 2025, filed 2026-02-20, accession 0001437749-26-004908, Item 7, page 36 of the course's PDF.

Cited by the page of its first character: page 36. By the page rule a miss; the chunk holds the figure.

Retrieval drawn





4 of 5

own key's chunk highest

2 of 5

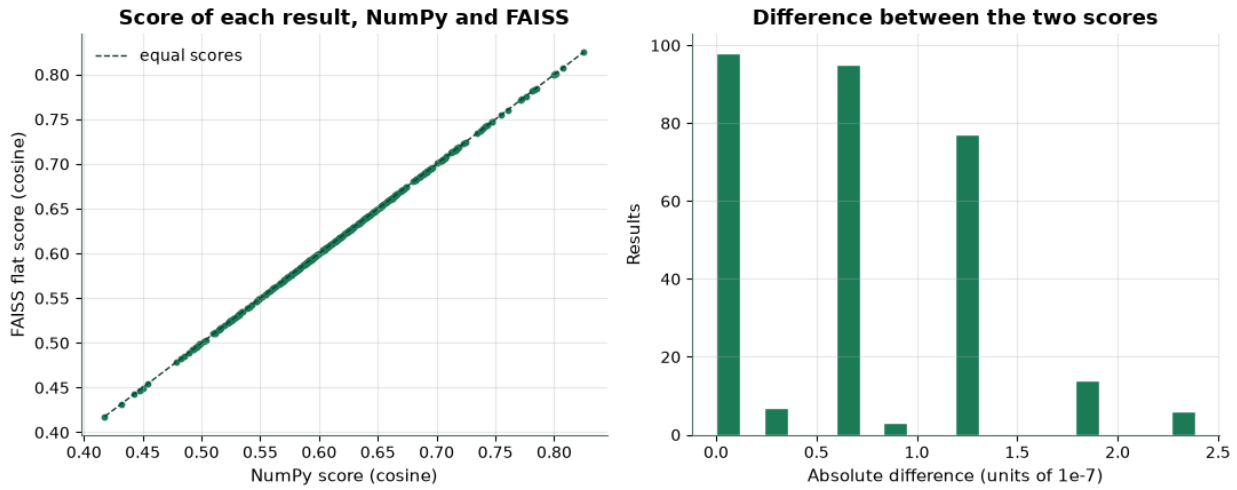
key chunk in the top 3

0.6802 = 0.6802

cosine = dot product

the chunk's vector times 5: dot product 3.4009, cosine 0.6802; the cosine measures direction only

FAISS returns the same chunks



300

results compared

True

same chunk, same rank

True

scores within 1e-6

Johnson, Douze, Jegou: Billion-scale similarity search with GPUs

Submitted to arXiv 2017-02-28 (arXiv 1702.08734). The FAISS repository asks that the GPU version cite it.

Sources: export.arxiv.org/api/query?id_list=1702.08734 and github.com/facebookresearch/faiss, read 2026-10-12.

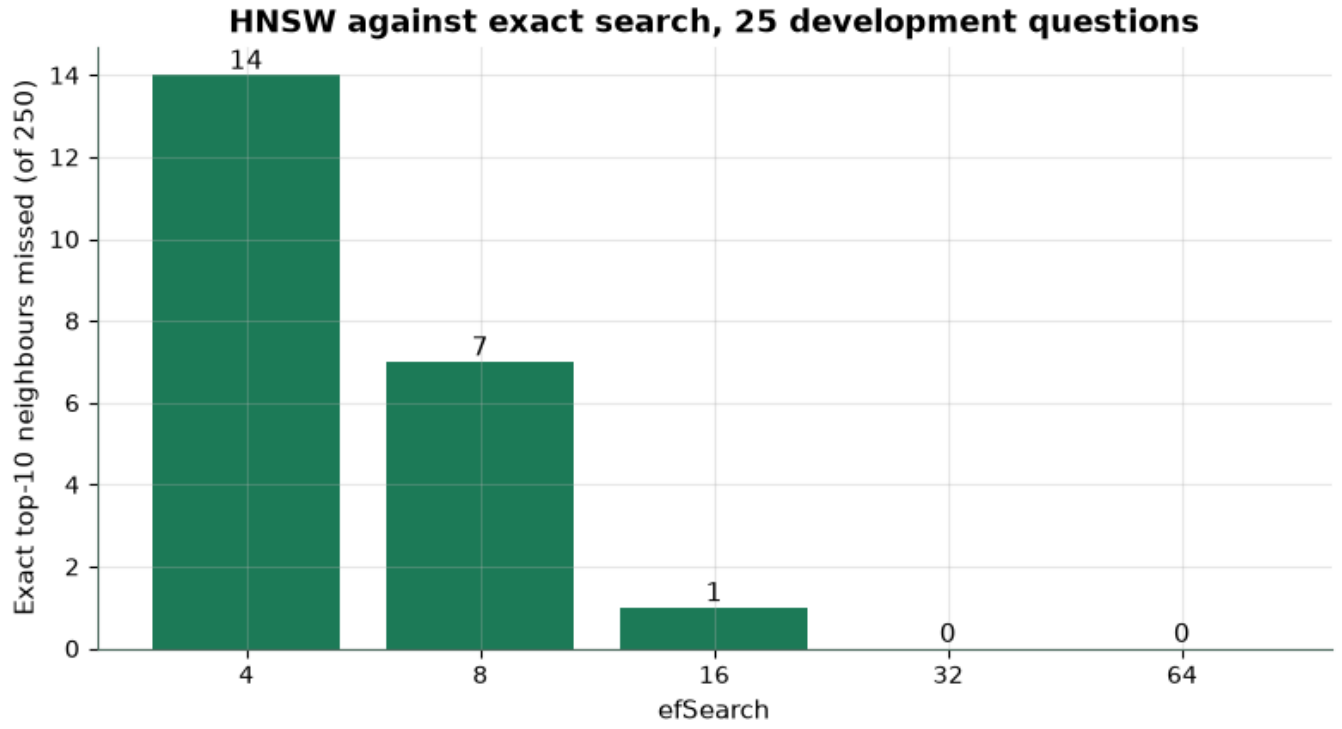
The paper itself was not read for this course.

faiss_index refuses rows that are not of length 1; the library would have accepted them.

How HNSW works

The problem	Exact search compares the question with every vector: one product here, a cost on every question at millions
The graph	Built before any search: each vector linked to M near neighbours, in layers, sparse on top, dense at the bottom
The search	Start at an entry point, move to the closer neighbour, keep the best efSearch candidates, stop when none improves
The miss	A true neighbour that no visited vector links to is never seen: that is the approximation
You control	M (links: larger graph, fewer misses) and efSearch (candidates: fewer misses, slower); both measured against exact

An approximate index, and what it gives up



Exact neighbours missed, of 250

efSearch 4	**14**
efSearch 8	**7**
efSearch 16	**1**
efSearch 32	**0**
efSearch 64	**0**

At this size the course uses exact search: one matrix product, nothing missed.

Chroma: named, not installed

27

requirements on Python 3.13 (chromadb 1.5.9)

14

already in this environment

13

would be new

Not installed here

`build, pydantic-settings, pybase64, onnxruntime, opentelemetry-exporter-otlp-proto-grpc, opentelemetry-sdk, pypika, overrides, importlib-resources, bcrypt, kubernetes, tenacity, mmh3`

Sources: pypi.org/pypi/chromadb/json and docs.trychroma.com/docs/overview/oss, read 2026-10-12.

Every idea of the course carries over: the chunks, the model and its revision, the manifest.

What an index was built from

index_manifest, MiniLM

documents	`boyd10k: 682fe12cd0c8...`
chunker	`size 512, overlap 64`
model	`minilm, revision 1110a24`
chunks	`1158`
matrix	`21f5de7f8705...`

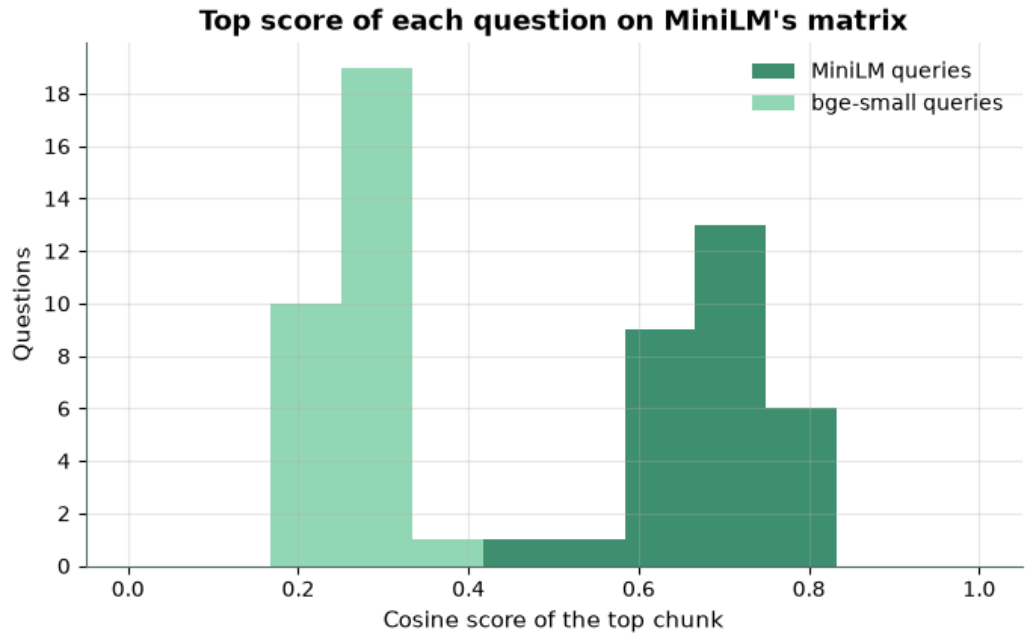
65 chunks changed by one letter

```
boyd10k_512_64_minilm.npy was made from other chunks:
re-chunk with {'overlap': 64, 'size': 512} or re-embed
```

The check stops before a single search.

Change the chunker, document, order, or model: the same file gives wrong answers with no error.

The wrong model's questions on MiniLM's matrix



Page hits do not collapse

MiniLM questions

top score 0.6921; key page for 16 of 28

bge-small questions

top score 0.2570; key page for 11 of 28

three random chunks per question: 0.8 of 28

40 different chunks in 90 slots; one returns 14 times

38% of slots on pages 34-43; 19 of 28 keys there

A hit count alone cannot tell a working index from a broken one; the score level and the manifest can.

Every setting of the day's functions

20 settings in eight signatures, read with inspect; 4 have a default, plus textkit.embed's batch_size

The settings with a default: what each controls, and what the course sets

`load_embedder`	`key`	`'minilm'`	which of the three models, at its pinned revision; course: minilm...
`faiss_index`	`kind`	`'flat'`	flat (exact) or hnsw (approximate); course: flat
`faiss_index`	`m`	32	HNSW links per vector; larger: a larger graph, fewer misses; cour...
`faiss_index`	`ef_search`	64	HNSW candidates kept per query; larger: fewer misses, a slower se...
`textkit.embed`	`batch_size`	64	texts the model runs at once; the last bits of an embedding depen...

The rest are required: the data, and k

a matrix, a chunk table, the question vectors, the model, a path: what the day builds, not a choice

k, results per question: set by hand on every day (10 today, 3 on day 5)

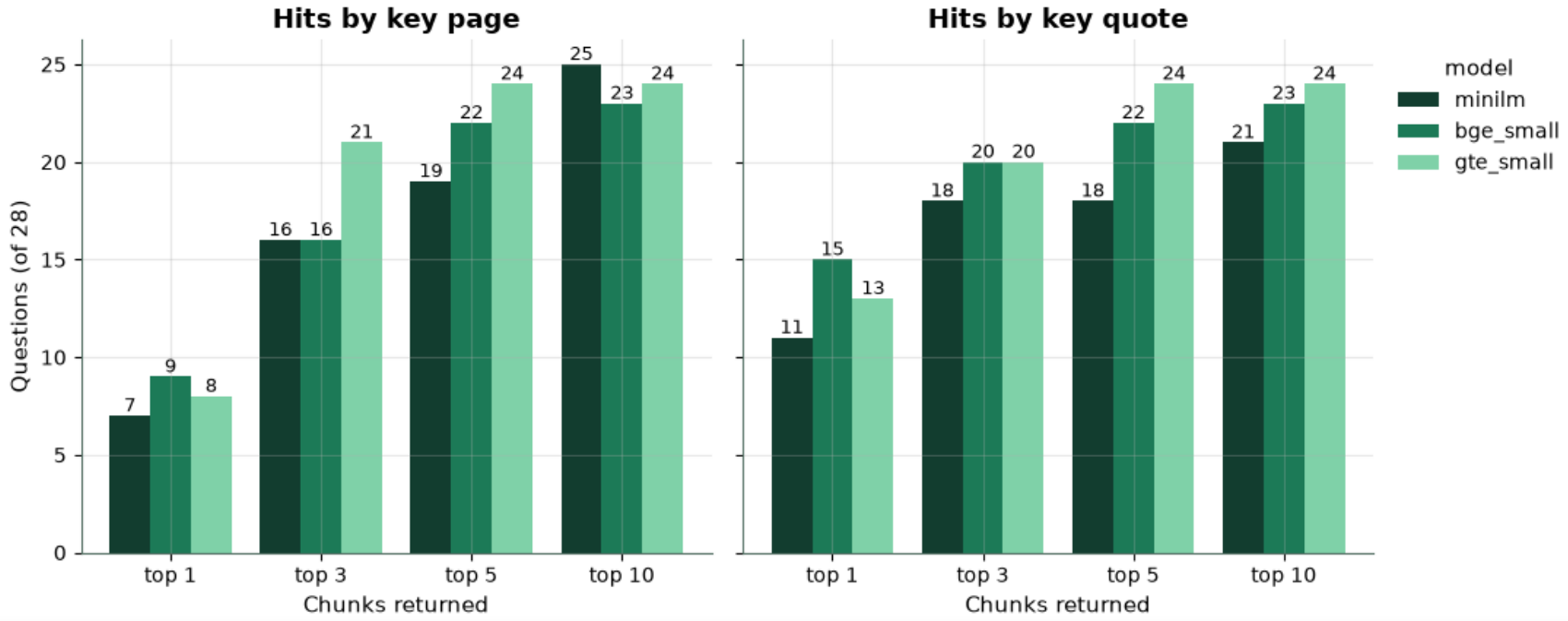
Three models on the five arc questions

Pages of the top three chunks, rank 1 first

question	key	minilm	bge_small	gte_small	models with key page
arc01	39	39, 39, 77	39, 77, 36	39, 77, 77	3
arc02	37	36, 19, 82	36, 48, 75	36, 48, 75	0
arc03	75	75, 75, 37	75, 75, 75	75, 37, 75	3
arc04	38	76, 19, 76	76, 19, 101	76, 38, 101	1
arc05	48	48, 48, 36	48, 48, 40	48, 48, 42	3

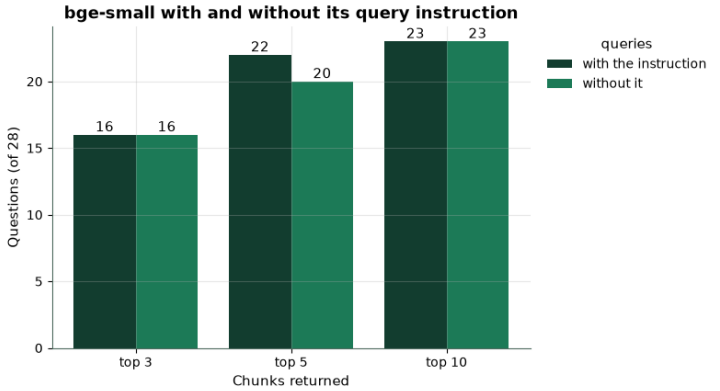
By the page rule no model has page 37 for arc02; all three return the chunk of slide 5 first.

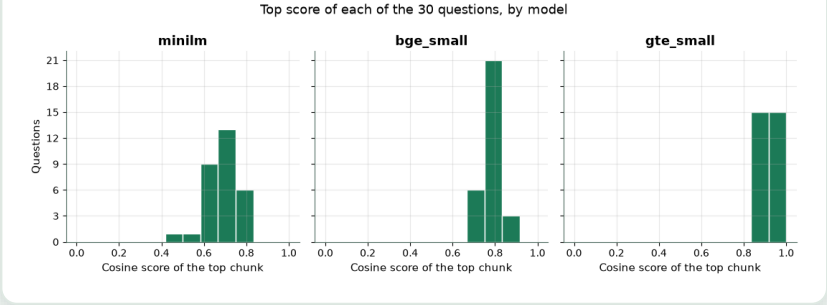
Hits, counted two ways

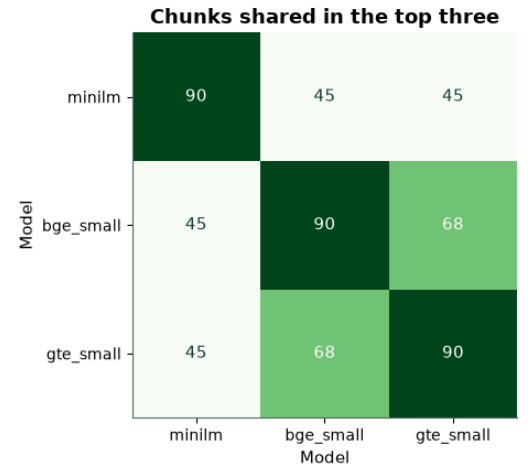


model	measure	top 1	top 3	top 5	top 10
minilm	key page	7	16	19	25
minilm	key quote	11	18	18	21
bge_small	key page	9	16	22	23
bge_small	key quote	15	20	22	23
gte_small	key page	8	21	24	24
gte_small	key quote	13	20	24	24

Instruction, scales, and agreement







Instruction, top 3 / 5 / 10: with 16 / 22 / 23; without 16 / 20 / 23

Mean top score: minilm 0.69, bge_small 0.78, gte_small 0.92: comparable inside one model only

Top-three chunks shared, of 90: MiniLM and bge-small 45, MiniLM and gte-small 45, bge-small and gte-small 68

The tables describe; they make no choice. The course keeps MiniLM.

Excel side by side, and Git

Excel side by side (tested in Excel)

`=SUMPRODUCT(A,B)/(SQRT(SUMSQ(A))*SQRT(SUMSQ(B)))`	cosine 0.6802; NumPy within 7e-16
`=SQRT(SUMSQ(B2:B385))`	length 1.000000
`=SUMPRODUCT($A$2:$A$385,B2:B385)`	= the cosine, unit vectors (4e-8)
`=RANK(B387,$B$387:$D$387)`	ranks 1, 2, 3 = NumPy's
`=SUMPRODUCT($A$2:$A$385,E2:E385), chunk x 5`	dot 3.4009; the cosine stays 0.6802

Git: index_manifest.json for MiniLM, then bge-small's: one file, the model and matrix fields; 25 tests.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the vectors go into the index as they are, so the inner product rewards long vectors.

Code that runs is not the same as code that is right.

What you can do now

- Check a committed matrix by fingerprint and a live sample
- Search exactly with NumPy; FAISS returns the same chunks
- Measure what an approximate index misses
- Record what an index was built from; refuse other chunks
- Say which measure a hit count uses

Week 1, Day 5 Case study: the first RAG pipeline